

Document Trap

Hallucination was a fake claim. Training Bias was a distorted picture. The third accuracy trap is different again.

You can drag in a PDF, paste a Google Doc, or upload a worksheet, and the AI will answer questions about it. It’s one of the most common ways teens use AI for school, and one of the easiest places to get a confident wrong answer. **The trap is thinking ‘uploaded’ means ‘fully read.’**

WHAT IT LOOKS LIKE

You upload your league’s 100–page rulebook and ask: *“How many fouls until I’m out of the game?”*

AI ANSWER

“Five fouls and you foul out.”

ALSO IN THE RULEBOOK, PAGE 47

“In tournament games, players foul out on the sixth foul.”

The AI pulled the standard foul limit and missed the tournament exception. The answer wasn’t made up. It was incomplete.

WHAT’S ACTUALLY HAPPENING

It would be nice if AI worked like this: you upload a document, the model reads the whole thing top to bottom, then answers your question with the whole document in mind.

That’s not always what happens. Sometimes the whole document fits in the context window. Often it doesn’t. And even when a lot fits, the system may search, summarize, or prioritize parts of the file rather than read it cover to cover.

There’s a name for what’s often happening: **retrieval**. When the system can’t load everything, it tries to retrieve the pieces that match your question. Done well, this lets you find a specific answer in a 50–page document in seconds. Done poorly, the wrong pieces get pulled, and the model answers from incomplete evidence.

Say you upload your basketball league’s 100–page rulebook and ask one question. The system can’t always load the whole thing, so it searches for the pieces that match.

How Chunks Get Picked

1

Your document gets split into chunks

Before you ever ask a question, the rulebook gets cut into smaller pieces, each a paragraph or two long. Maybe 80 chunks total. The chunks sit in a database, waiting.

2

Each chunk gets a meaning vector

Same idea you saw in the Embeddings lesson. Every chunk gets translated into a meaning profile, a list of numbers that captures what the chunk is about. The chunk about technical fouls ends up close in meaning to other rulebook chunks about penalties, referee calls, and player conduct.

3

Your question becomes a vector too. The closest chunks get loaded.

Now you ask: “Can a player call a timeout during a free throw?” Your question gets its own meaning vector. The system finds the chunks whose vectors are closest. Those chunks (not the whole rulebook) get loaded. The model answers based on what got retrieved.

WHAT GOT LOADED

The model never saw the whole rulebook.

It answered from only the chunks that matched your question. If the chunk that mattered didn’t get pulled, the answer still sounds confident. It is just missing what it never loaded.

You can’t always tell what got retrieved. But you can steer it. Five moves help.

The five moves

1

Point to the section by name

Include keywords from the document in your question. The system uses them to find matching chunks.

2

Ask one question at a time

Multi-part questions force the system to retrieve different chunks for different parts. Break it up. One focused question per turn gives retrieval the best shot.

3

Paste the section directly into your prompt

If you already know the paragraphs that matter, paste them into your message. That puts them in context with no retrieval lottery.

4

Upload only what’s relevant

Instead of the whole rulebook, upload just the chapter you need. Fewer chunks competing for retrieval slots means the right ones are more likely to make the whiteboard.

5

Ask the AI to quote

Add ‘quote the exact section you’re basing this on’ to your question. If the quote is missing or doesn’t match the document, retrieval probably failed.

All five share one idea: make the right chunks easy to find. The more specific your question, the more reliable the answer.



Uploaded does not mean understood. Confident does not mean complete. Asking for evidence is the skill.